

Մաս 1. Ասիմպտոտիկ Նշանակումներ և Մաստեր Մեթոդ

1.1 Սահմանի մեթոդը (Limit Method)

Եթե ունենք $f(n)$ և $g(n)$ ֆունկցիաները, հաշվում ենք սահմանը՝

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

- $L=0 \Rightarrow f(n)=o(g(n))$ և $f(n)=O(g(n))$: (g -ն աճում է ավելի արագ):
- $L=c>0 \Rightarrow f(n)=\Theta(g(n))$: (Աճում են նույն տեմպով):
- $L=\infty \Rightarrow f(n)=\omega(g(n))$ և $f(n)=\Omega(g(n))$: (f -ն աճում է ավելի արագ):

1.2 Master Method (Մաստեր Մեթոդ)

$T(n) = aT(n/b) + f(n)$ ռեկուրսիաների համար.

Համեմատում ենք $f(n)$ – $n^{\log_b a}$ -ի հետ.

- **Case 1:** Եթե $f(n)=O(n^{\log_b a - \epsilon}) \Rightarrow T(n)=\Theta(n^{\log_b a})$
- **Case 2:** Եթե $f(n)=\Theta(n^{\log_b a}) \Rightarrow T(n)=\Theta(n^{\log_b a} \log n)$
- **Case 3:** Եթե $f(n)=\Omega(n^{\log_b a + \epsilon}) \Rightarrow T(n)=\Theta(f(n))$

Մաս 2. Սորտավորման Ալգորիթմներ (Python)

2.1 Quicksort (Lomuto Partition)

- **Բարդությունը:** Միջինում $O(n \log n)$, Վատագույն դեպքում $O(n^2)$ (Երբ սորտավորված է):
- **Հիշողություն:** $O(\log n)$ (In-place):
- **Կայունություն (Stable):** Ոչ:

```
def partition(arr: list, low: int, high: int) -> int:
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i + 1

def quicksort(arr: list, low: int, high: int):
    if low < high:
        pi = partition(arr, low, high)
        quicksort(arr, low, pi - 1)
        quicksort(arr, pi + 1, high)
```

2.2 Merge Sort

- **Բարդությունը:** Միշտ $O(n \log n)$:
- **Հիշողություն:** $O(n)$ (Ոչ in-place):
- **Կայունություն (Stable):** Այո:

```
def merge_sort(arr: list):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0
        # Միաձուլում ենք երկու կեսերը
        while i < len(L) and j < len(R):
            if L[i] <= R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1

        # Ավելացնում ենք մնացորդները
        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1
        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
```

2.3 Heapsort

- **Բարդությունը:** Միշտ $O(n \log n)$:
- **Հիշողություն:** $O(1)$ (In-place):
- **Կայունություն (Stable):** Ոչ:

```
def max_heapify(arr: list, n: int, i: int):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
```

```

•     if right < n and arr[right] > arr[largest]:
•         largest = right
•
•     if largest != i:
•         arr[i], arr[largest] = arr[largest], arr[i]
•         max_heapify(arr, n, largest)
•
• def heapsort(arr: list):
•     n = len(arr)
•     # 1. Կառուցում ենք Max-Heap (Build-Max-Heap)
•     for i in range(n // 2 - 1, -1, -1):
•         max_heapify(arr, n, i)
•
•     # 2. Հեռացնում ենք արմատները մեկ առ մեկ (Extract-Max)
•     for i in range(n - 1, 0, -1):
•         arr[i], arr[0] = arr[0], arr[i] # Փոխանակում
•         max_heapify(arr, i, 0)

```

Մաս 3. Տվյալների Գծային Կառուցվածքներ (Python)

3.1 Stack (Ստեկ)

- **Սկզբունք:** LIFO (Last In, First Out)
- **Բարդություն:** Push $O(1)$, Pop $O(1)$
- Python-ում ստեկը շատ հարմար է իրականացնել սովորական list-ի միջոցով:

```

class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return "Stack is empty"

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        return None

    def is_empty(self):
        return len(self.stack) == 0

```

3.2 Queue (Հերթ)

- **Սկզբունք:** FIFO (First In, First Out)
- **Բարդություն:** Enqueue $O(1)$, Dequeue $O(1)$
- Python-ում արդյունավետ հերթի համար օգտագործվում է collections.deque-ն, քանի որ սովորական list-ից սկզբից տարր հեռացնելը $O(n)$ է:

```
from collections import deque

class Queue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.popleft()
        return "Queue is empty"

    def is_empty(self):
        return len(self.queue) == 0
```

3.3 Singly Linked List (Միակապ ցուցակ)

- **Բարդություն:** Access $O(n)$, Insert/Delete (սկզբում) $O(1)$, ավելացում վերջում $O(n)$ (առանց tail ցուցիչի):

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return

        last = self.head
        while last.next:
```

```

        last = last.next
        last.next = new_node

def display(self):
    current = self.head
    elements = []
    while current:
        elements.append(str(current.data))
        current = current.next
    print(" -> ".join(elements))

```

4.1 Counting Sort (Հաշվարկային սորտավորում)

Ի տարբերություն նախորդների, սա **համեմատական չէ** (non-comparison sort): Այն աշխատում է միայն ամբողջ թվերի հետ որոշակի միջակայքում $[0, k]$:

- **Բարդությունը:** $O(n+k)$, որտեղ k -ն թվերի միջակայքն է:
- **Հիշողություն:** $O(n+k)$:
- **Կայունություն (Stable):** Այո:

```

def counting_sort(arr: list):
    if not arr:
        return arr
    max_val = max(arr)
    count = [0] * (max_val + 1)
    output = [0] * len(arr)

    # Հաշվում ենք ամեն տարրի քանակը
    for x in arr:
        count[x] += 1

    # Կուտակային գումար (նշում ենք դիրքերը)
    for i in range(1, len(count)):
        count[i] += count[i-1]

    # Կառուցում ենք ելքային գանգավածը (հետևից առաջ՝ կայունության համար)
    for i in range(len(arr) - 1, -1, -1):
        output[count[arr[i]] - 1] = arr[i]
        count[arr[i]] -= 1

    return output

```

4.2 Insertion Sort (Ներդրման սորտավորում)

Աշխատում է այնպես, ինչպես խաղաթղթերն են դասավորում ձեռքում:

- **Բարդությունը:** $O(n^2)$ (Միջին և Վատագույն), բայց $O(n)$, եթե զանգվածը համարյա սորտավորված է:
- **Հիշողություն:** $O(1)$ (In-place):
- **Կայունություն (Stable):** Այո:

```
def insertion_sort(arr: list):  
    n = len(arr)  
    for i in range(n):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key
```

4.3 Selection Sort (Ընտրովի սորտավորում)

Ամեն քայլում գտնում է մնացած մասի ամենափոքր տարրը և բերում սկիզբ:

- **Բարդությունը:** Միշտ $O(n^2)$ (անկախ տվյալների վիճակից):
- **Հիշողություն:** $O(1)$ (In-place):
- **Կայունություն (Stable):** Ոչ (փոխանակումները կարող են խախտել կարգը):

```
def selection_sort(arr: list):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

4.4 Bubble Sort (Պղպջակային սորտավորում)

Հարևան տարրերը համեմատվում են և տեղերով փոխվում, եթե սխալ կարգով են:

- **Բարդությունը:** $O(n^2)$:
- **Հիշողություն:** $O(1)$ (In-place):
- **Կայունություն (Stable):** Այո:

```
def bubble_sort(arr: list):  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(n - i - 1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
                swapped = True  
        if not swapped:  
            break
```